AMS SCENARIO 3 — ENHANCEMENT · TECHNICAL BRIEF

How the AI chooses what it reads

The relevance funnel behind S3, the numbers that justify it, and an honest comparison with GitLab Duo.

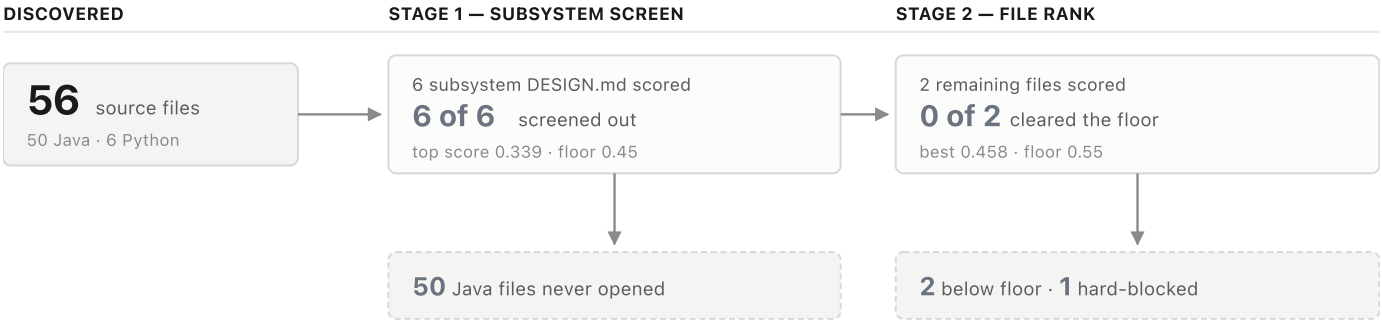
MapleSure Insurance demo build · All figures measured live against CR-2026-041 on 27 July 2026 · Local embedding backend

> **IF YOU ARE ASKED "HOW DOES IT RANK?" — SAY THIS**
>
> *"We never paste the repo into the prompt. First we score each subsystem's own design document against the change request and discard whole subsystems that don't match — the AI reads the architecture docs before it opens a single source file. Then we rank only what survived, and send a handful. On this change request that's 4 files out of 56, and the console shows you every score and everything we threw away."*

The funnel, with real counts

Figure 1 · What happens to 56 files

CR-2026-041 (new top coverage tier) against the MapleSure mockapp target.



THE CONTRACT LANE — BYPASSES SCORING ENTIRELY

4 **files sent to the model**

app.py · core/db.py · core/models.py — always included, never scored — plus core/coverage.py, sent empty because the CR creates it

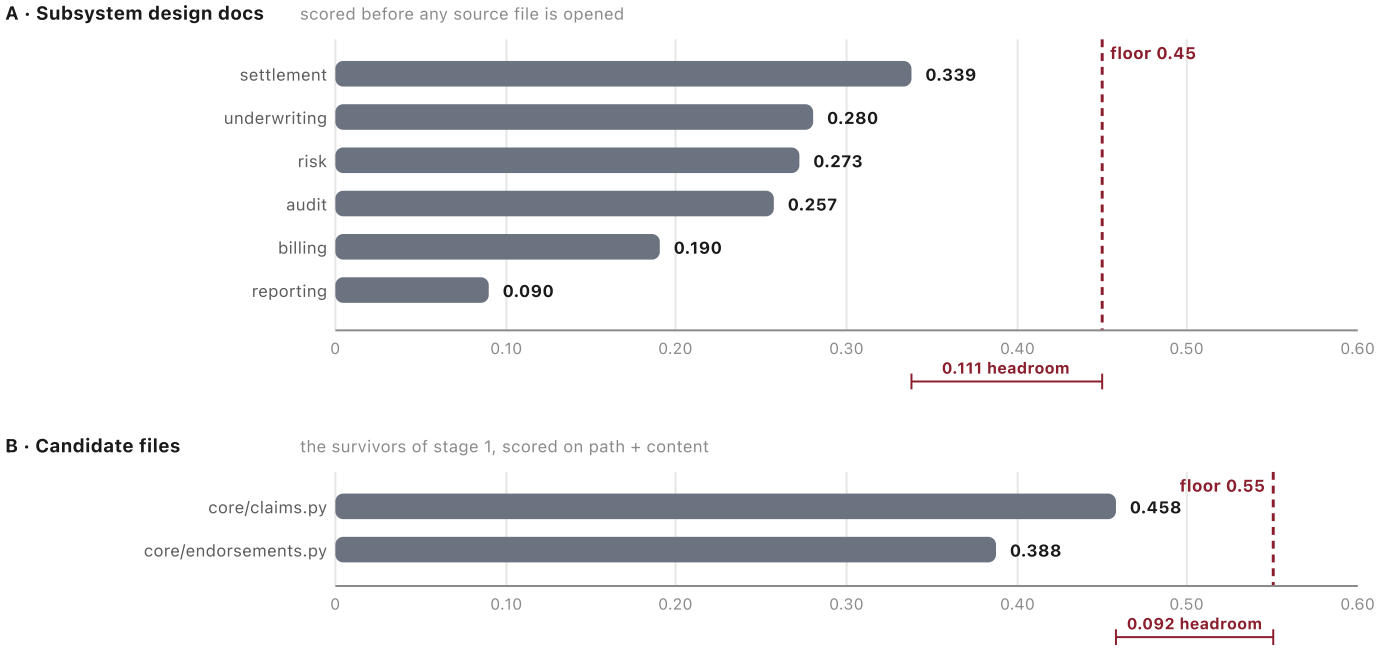
The ranker's job on this CR is exclusion, not selection. Nothing cleared the floor; the four files that went were the ones the target's core-file contract guarantees. That is the design working as intended — ranking decides what *else* the model may see, and it declined to add anything.

Why the thresholds are defensible

The floors are not taste. They sit above the highest score any known-irrelevant file or subsystem actually achieves, measured against this corpus — so a decoy would have to beat its own ceiling by a wide margin to get in.

Figure 2 · Measured cosine similarity vs. the cut-off

Every scored item on this change request. Same unit and scale in both panels; each panel carries its own floor.



The gap is the argument. `core/claims.py` is the hardest case in the corpus — same insurance vocabulary as the CR, genuinely adjacent, but not what the change is about. It scores 0.458 against a 0.55 floor. The nearest decoy subsystem, *settlement*, tops out at 0.339 against 0.45. Both are comfortably clear, and `tests/test_s3_relevance.py` asserts they stay that way.

> **ANSWER IF CHALLENGED ON THE NUMBERS**
>
> These floors are calibrated to *this* corpus, and the brief says so. The honest claim is that the **mechanism** generalises and the **constants** get re-measured per repository — the same way you'd re-tune any detector. Do not claim the numbers transfer.

Two guardrails that are not ranking at all

Worth separating, because they answer the question relevance scoring cannot: *what if the ranker is wrong?*

Core files bypass scoring entirely

The files a CR provably needs are declared per target and always included — and, more importantly, **required back in the model's response**. `verify_core_recall()` runs against the LLM's actual returned file set, not just the prompt. A generation that quietly drops a required file is rejected, not shipped. Duo suggests; this one refuses.

The top-scoring file in the whole corpus is hard-blocked

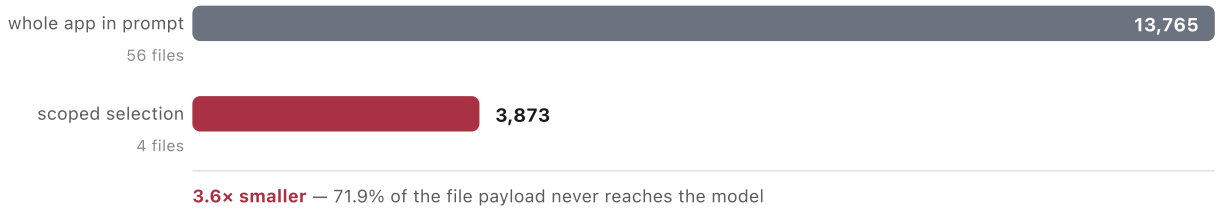
The best line in the deck. `mockapp/core/seed.py` would score higher against this CR than any other candidate — it is the file most densely packed with `Policy` field names. It is also the one file that must never be edited, because it constructs `Policy(...)` with six positional arguments that the whole demo depends on. So it is excluded structurally, by `NEVER_EXTRA`, regardless of score.

> **SAY IT LIKE THIS**
>
> *"The highest-relevance file in the repository is one we refuse to let the model touch. Relevance and safety are different questions, and we answer them separately."*

What scoping actually costs — and saves

Figure 3 · File payload sent to the model

Estimated tokens of file content, CR-2026-041. Prompt scaffolding is identical on both sides and excluded.



Use the real multiple, not a rounder one. 3.6× is what this CR measures, because the four files that *do* go are the large ones. The compelling number is not the ratio — it is that the ratio *widens as the repository grows*, since the selection stays at a handful while the denominator climbs.

> **DO NOT OVERSELL THIS SLIDE**
>
> On the Spring ClaimsPortal target the pool is 8 files and scoping has nothing to remove — selection is the whole repo. If someone asks whether it always saves tokens, the answer is no: **on a small codebase it saves nothing, and that is correct behaviour.** The funnel earns its keep at scale, not on a toy.

Large repositories add a cheaper tier first

For a GitLab-backed target the funnel gains a stage that runs *before* any content is fetched: list every path (cheap), rank the bare path segments against the CR with TF-IDF, keep the top 20, and only then fetch those files' contents. A twenty-repository account never costs more than a handful of small HTTP GETs, whatever any individual repo's size.

Note the deliberate asymmetry, because it reads as an inconsistency if unexplained: path pre-ranking uses TF-IDF, *not* embeddings. A string like `src billing export py` is not natural language, and a sentence-embedding model is the wrong instrument for it. Prose gets embeddings; path tokens get lexical matching.

Against GitLab Duo — where we are genuinely stronger

DIMENSION	THIS BUILD	GITLAB DUO	EDGE
Auditability of retrieval	Candidate pool size, every per-file score, and the complete screened-out list are rendered in the console for each run.	Retrieval is opaque. You see the answer, not the shortlist or why.	OURS
What drives selection	Your own `DESIGN.md` scope declarations screen first. Documented architecture boundaries become the AI's boundaries.	Code and embedding similarity. Your architecture docs are not a control surface.	OURS
Output enforcement	Core-file recall, exact API names, error-message wording and backward-compat are validated against the model's response; violations are rejected.	Suggestion-shaped. Review is the human's job.	OURS
Runs air-gapped	Local embedding model, no API key, no network at index or query time; lexical fallback if the vector store is unavailable.	SaaS-first. Self-managed exists but the AI path still reaches out.	OURS
Cost transparency	Scoped-vs-naive token panel on every beat.	Seat-priced; per-change context cost is not surfaced.	OURS
Repository scale	Thresholds hand-measured against a 56-file corpus. Re-calibration is a real per-repo task.	Continuous indexing across large real repositories, maintained for you.	DUO
Developer surface	A standalone console. One workflow, deliberately.	In the IDE, in merge requests, in CI, where developers already are.	DUO
Zero-setup value	Stage 1 does nothing without `DESIGN.md` files; undocumented repos fall back to stage 2 alone.	Works on any repo on day one, no documentation prerequisite.	DUO

> THE POSITIONING LINE
>
> *"Duo is the better general-purpose assistant, and we would not try to out-build it. What we've built is the governed path: every file the model saw, every file it was denied, and a generation that gets rejected when it breaks the contract. In a regulated change process that difference is the product."*

The prerequisite, framed correctly

The `DESIGN.md` dependency will come up. Do not defend it as free — frame it as the mechanism: *this approach rewards teams who document their boundaries, and turns those documents into an enforced control rather than a wiki page nobody reads.* That is a roadmap conversation, not a weakness.

Likely questions, short answers

QUESTION	ANSWER
"How did you pick 0.55?"	Measured. The closest irrelevant file scores 0.458; the floor sits above it with headroom, and a test asserts it.
"What if it excludes something needed?"	Required files never go through ranking — they are contractual, and their presence is verified in the model's *output*.
"Does this scale to our monorepo?"	The funnel does; the constants get re-measured. For GitLab targets the path pre-rank means repo size costs almost nothing.
"Why not just use a bigger context window?"	Cost and precision. More irrelevant context measurably degrades edit accuracy — and you still cannot show anyone *why* a file was used.
"Is the embedding model sending our code anywhere?"	No. It runs locally, on the machine, with no API key and no network call at index or query time.
"Why did nothing get selected on this run?"	Because nothing deserved to. The four contract files were sufficient — exclusion is the ranker doing its job.

Method. All figures produced by running this repository's own `s3_enhancement/relevance.py` against `mockapp/crs/CR-2026-041.md` on 27 July 2026, with the local embedding backend active (not the lexical fallback). Token counts use the repository's ~4-characters-per-token estimator and are explicitly estimates; a billed figure always comes from the provider's reported usage. Scores drift slightly as the corpus changes — re-run before quoting these in a later deck. Chart colours were validated for the light print surface; the neutral grey marking excluded items is a deliberate recessive encoding, and every bar carries a direct numeric label so nothing depends on colour alone.